# AB2DETECT: A Hallucination Detection and Correction System for RAG-Based AI Pipelines

Dr. Ramakrishnan Varadharajan*, Abhishek D Nagoor*, Abubakar Khan*
*School of Computer Science and Engineering, RV University, Bengaluru, Karnataka, India
ramakrishnanv@rvu.edu.in, abhishekdnagoorbtech24@rvu.edu.in, abubakarkhanbtech24@rvu.edu.in

***Abstract*—Large language models used inside Retrieval-Augmented Generation (RAG) pipelines still hallucinate—they produce content that is not supported by, or directly contradicts, the retrieved context, even when the relevant information is present in that context. This paper describes AB2DETECT, a hallucination detection and correction system built during a research internship at the School of Computer Science and Engineering, RV University. It builds on the open-source LettuceDetect framework, which performs token-level span classification with ModernBERT (149M parameters, 4,096-token context window) and EuroBERT (8,192-token context, seven European languages), labeling every answer token as either supported or hallucinated relative to the retrieved context. Beyond the core detector, this work adds five original components: (1) an Auto-Correction module that locates hallucinated spans and rewrites them using context-grounded evidence; (2) a dashboard comparing hallucination rates across four free, locally runnable LLMs (Mistral-7B, Zephyr-7B, Falcon-7B, and LLaMA-2-7B); (3) a small, manually annotated cricket-domain pilot dataset with gold-standard hallucination spans; (4) an exploratory Kannada-language hallucination evaluation—to our knowledge the first of its kind on RAG outputs in an Indian language; and (5) a Chrome extension that checks for hallucinations directly on a webpage without a paid API key. On the RAGTruth benchmark, ModernBERT-Base reaches a span-level F1 of 68.4% on the Question Answering task, 7.6 points above a GPT-4 LLM-judge baseline (60.8%), while running roughly $15\times$ faster on CPU and at zero per-query cost; ModernBERT-Large reaches 72.1% F1. On a five-sample cricket pilot set the system scores 100% F1; owing to the sample size, we treat this as a qualitative case study and not as evidence of domain generalization. On Data-to-Text generation—the most structurally grounded of the three task types—ModernBERT-Large reaches 75.2% F1. Taken together, these results suggest that lightweight, task-specific encoder models can match or exceed costly LLM-judge pipelines for hallucination detection in production RAG systems, while remaining fast and cheap enough for real-time deployment.**

***Index Terms*—Large Language Models, Hallucination Detection, Retrieval-Augmented Generation, ModernBERT, EuroBERT, Token Classification, Span Detection, RAGTruth, Natural Language Processing, Low-Resource Languages.**

## I. Introduction

### A. Background

Large language models such as GPT-4, LLaMA-2, Mistral, and Falcon have reshaped what is possible in natural language processing. Trained on enormous corpora, they perform remarkably well at understanding language, reasoning through problems, and holding a conversation. However, one failure mode has proven persistent: these models hallucinate. They confidently state content that is incorrect, fabricated, or inconsistent with the information they were given.

Retrieval-Augmented Generation (RAG) is one of the more widely used strategies for addressing this in production systems [11]. A RAG pipeline retrieves relevant document chunks from a knowledge base using the user's query, appends that evidence to the query, and hands both to an LLM, which then generates an answer grounded in the retrieved material. In theory this ties the model's output to something verifiable. In practice, models still hallucinate inside RAG pipelines—either contradicting information stated plainly in the retrieved context (intrinsic hallucination) or introducing details that are not present in the context at all (extrinsic hallucination). Either way, it undermines the core purpose of RAG: that a generated answer can be traced back to real sources.

### B. Motivation

Undetected hallucination in deployed AI systems can have serious consequences. In medicine, a model that misstates a drug dosage or a contraindication could harm a patient. In legal work, citing a case that does not exist or misquoting a statute could lead to an unjust outcome. In financial advising, a fabricated statistic can trigger a costly decision. Even where the stakes are lower, hallucination erodes user trust in the system.

Existing detection approaches are poorly suited to production use. Using a large model like GPT-4 as a judge is expensive (roughly US$0.02 per query) and slow (2–10 seconds of latency), which rules out real-time integration for most applications. Traditional BERT-based classifiers are limited to a 512-token context (about 380 words), which is insufficient for realistic RAG contexts that span several retrieved passages. And sentence- or document-level methods, such as consistency sampling, cannot identify which specific words in an answer are problematic. AB2DETECT was developed to meet the need for a detector that is accurate, fast, inexpensive, and precise enough to identify the exact tokens where a hallucination occurs.

### C. Problem Statement

Given a retrieved context document $C$, a user question $Q$, and an LLM-generated answer $A$, this work builds a system that (1) identifies, at the token and span level, which parts of $A$ are not supported by $C$; (2) automatically corrects those hallucinated spans using information drawn from $C$; and (3) makes this capability accessible for real-time monitoring across multiple models, input modalities, and languages. That breaks down into three sub-problems: the core detection problem of achieving token-level classification accuracy competitive with LLM-judge baselines; the domain generalization problem of preliminarily probing whether a detector trained on general-purpose RAG data transfers to a specialized domain and to

a non-English language, using small pilot studies; and the usability problem of packaging all of this so that it requires no paid API key and can run in a browser.

### D. Objectives

The project set out to: (i) build a token/span-level hallucination detector on top of LettuceDetect's ModernBERT and EuroBERT backbones, competitive with LLM-judge baselines on RAGTruth; (ii) build an Auto-Correction module that replaces detected hallucinated spans with context-grounded content; (iii) build an interactive application that integrates four free, open-source LLMs with real-time span highlighting and no API-key requirement; (iv) construct and evaluate an original cricket-domain pilot dataset with manually annotated gold spans; (v) evaluate detection performance on Kannada text; (vi) build a Chrome extension for browser-integrated detection; and (vii) compare hallucination rates across several LLMs through an analytical dashboard.

### E. Scope

AB2DETECT works over text (direct input) and structured question-answering (QA) pairs (dataset evaluation); over English (primary) and Kannada (secondary, evaluation only); over the general-purpose RAGTruth benchmark and the original cricket-domain pilot dataset; and over four locally runnable LLMs (Mistral-7B, Zephyr-7B, Falcon-7B, LLaMA-2-7B). Its detection is limited to answer faithfulness relative to the retrieved context—it does not perform independent real-world fact-checking—and it does not support streaming token generation.

### F. Novelty and Contributions

The core detector is adopted from LettuceDetect [1]; its ModernBERT/EuroBERT token-classification architecture, input construction, and training recipe are *not* claimed as novel, and LettuceDetect performs detection only. The elements that are new in AB2DETECT are: (1) a **detect–correct–verify pipeline**: an Auto-Correction module that locates hallucinated spans, rewrites them using context-grounded evidence, and reruns the detector to confirm that no new hallucination was introduced; (2) a **multi-LLM faithfulness dashboard** reporting hallucination rate, span-level F1, and latency for four locally runnable LLMs; (3) a small **cricket-domain pilot dataset** with gold hallucination spans, used as a qualitative case study (Section V-E); (4) an **exploratory Kannada evaluation** using character-level matching, for which, to our knowledge, no comparable evaluation has been reported; and (5) a **key-free Chrome extension** backed by a local FastAPI server that highlights hallucinated spans without a paid API key. In addition, we provide a systematic evaluation of the detector across the three RAGTruth task types, together with a CPU-based latency and cost comparison against LLM-judge and NLI-based baselines.

## II. Literature Review

Twelve papers were surveyed to establish the state of the art in hallucination detection, RAG architecture, and encoder-based NLP.

**RAGTruth** [2] is the primary evaluation benchmark used in this work. It contains human-annotated hallucination spans produced by five LLMs (GPT-4, GPT-3.5-Turbo, LLaMA-2-7B, LLaMA-2-13B, and Mistral-7B) across three task types—Question Answering, Text Summarization, and Data-to-Text (D2T) generation—with inter-annotator agreement verified via Cohen's $\kappa$, making it the closest available approximation of a gold standard for RAG hallucination evaluation.

**LettuceDetect** [1] is the detection engine underlying AB2DETECT. It fine-tunes ModernBERT and EuroBERT for token-level sequence labeling on RAGTruth. The key design choice is how the input is built: context, question, and answer are concatenated as `[CLS] C [SEP] Q [SEP] A [SEP]`, and only the answer tokens get classification labels—context and question tokens are masked out of the loss (label $-100$ in PyTorch). This allows every answer token to attend bidirectionally over the whole context window, which is what makes faithfulness checking over long documents possible.

ModernBERT updates the original BERT architecture with Rotary Positional Embeddings (RoPE) and Flash Attention 2, and was trained on roughly two trillion tokens of contemporary English text. Where BERT relies on fixed absolute positional embeddings capped at 512 tokens, RoPE encodes relative token distances and extrapolates well beyond that, giving ModernBERT a 4,096-token window. Flash Attention 2 cuts attention memory complexity from $O(n^2)$ to $O(n)$, which makes long-sequence inference practical. ModernBERT comes in base (149M parameters) and large (395M parameters) variants.

**EuroBERT** [3] extends the ModernBERT architecture to seven European languages (English, German, French, Spanish, Italian, Portuguese, and Dutch) with a doubled 8,192-token window, via an extended RoPE configuration. The multilingual LettuceDetect variant fine-tunes EuroBERT on language-specific RAGTruth data and has reportedly outperformed GPT-4.1-mini by up to 17 F1 points on non-English tasks.

**SelfCheckGPT** [4] takes a sampling-based, zero-resource approach: it generates several completions for the same prompt and flags facts that vary across samples as likely hallucinations. This avoids the need for labeled data, but it is expensive (5–10 LLM calls per query), works only at sentence granularity, and misses cases where a model hallucinates the same wrong fact consistently.

**AlignScore** [5] fine-tunes a RoBERTa-based Natural Language Inference (NLI) model to score factual consistency between a claim and its source. Like most NLI-based methods, it is limited to a 512-token context and to sentence-level verdicts, and it tends to struggle with negation.

**FActScore** [6] breaks generated text into atomic facts and verifies each one independently, typically using an LLM—giving fine-grained results, but at the cost of being LLM-dependent and slow.

**HADES** [12] detects hallucinated named entities via NER followed by verification. It is efficient, but in RAGTruth, entity-level hallucinations account for only about 40% of annotated error spans, so a purely entity-based approach misses the majority of hallucination types, including numerical, relational, and causal errors.

**TruthfulQA** [9] is widely cited for measuring whether models mimic human falsehoods, but it evaluates world-knowledge accuracy rather than faithfulness to a retrieved RAG context, which limits how directly it applies here.

**Ji et al.'s survey** of hallucination in natural language generation [7] provides the taxonomic backbone for this project's framing—characterizing hallucination as a multi-cause phenomenon spanning intrinsic and extrinsic errors—but it stops short of proposing an actual detection system.

**PubMedQA** [8], along with the foundational BERT paper [10] and the original RAG paper [11], round out the survey: the former motivates the domain-generalization question, which the cricket and Kannada pilot studies here only begin to explore, and the latter two ground the encoder architecture and retrieval-augmented paradigm that AB2DETECT builds on.

Table I summarizes the comparison. Taken together, the literature points to a clear gap: encoder-based detectors are fast and cheap but have historically been context-limited, while LLM-judge and sampling-based approaches handle long contexts but are slow, costly, and typically coarse-grained. AB2DETECT is positioned to close that gap by pairing a long-context encoder (ModernBERT/EuroBERT) with token-level supervision.

## III. Methodology

### A. Limitations of Prior Approaches

Four broad methodologies came before AB2DETECT, each with its own shortcoming. The LLM-judge approach prompts a powerful model like GPT-4 to render a verdict on an answer against its context—accurate, but it costs roughly US$0.02 per query, adds 2–10 seconds of latency, and typically only gives a sentence- or document-level verdict rather than pinpointing the actual hallucinated tokens. Traditional BERT-based NLI classification (e.g., AlignScore) is limited to a 512-token window, which is too short for realistic multi-passage RAG contexts. Sampling-based consistency checking (e.g., SelfCheckGPT) does not need labeled data but multiplies inference cost by requiring 5–10 LLM calls per query, and cannot catch hallucinations that stay consistent across samples. Entity-level detection (e.g., HADES) is cheap to run but structurally blind to non-entity errors—numerical, relational, and causal hallucinations—which together make up the majority of annotated RAGTruth spans. Table II lays out these trade-offs against the approach proposed here.

### B. Proposed System: AB2DETECT

AB2DETECT is a modular, extensible hallucination detection and correction system made up of a core detector and five additional functional modules, all integrated into a single deployable Streamlit application (Fig. 1). The overall flow: a question and a context are fed into an LLM to generate a candidate answer, which then goes to the core detection module; from there, detected spans can flow onward into the Auto-Correction module, the multi-model dashboard, or be surfaced through the Chrome extension.

**Core Detection Module.** Detection happens in six steps. (1) *Input construction*: question $Q$, context $C$, and answer $A$ are concatenated as `[CLS] C [SEP] Q [SEP] A [SEP]`. (2) *Tokenization*: the sequence is tokenized with the ModernBERT tokenizer, keeping a mapping from token to character position. (3) *Label masking*: only answer tokens get supervision; context and question tokens are masked with label $-100$ during training and evaluation. (4) *Encoder forward pass*: the full sequence passes through ModernBERT's bidirectional self-attention, so every answer token's representation reflects information from the entire context and question. (5) *Classification head*: a linear layer maps each answer token's representation to a two-way logit (supported vs. hallucinated), converted to a probability with softmax. (6) *Span aggregation*: consecutive tokens predicted as hallucinated are merged into spans, each carrying a confidence score (mean token probability) and character-level start/end offsets in the original answer.

**Auto-Correction Module.** Given the spans from the step above, the correction pipeline (i) identifies hallucinated spans along with their confidence and position; (ii) re-queries the context using string matching over numbers, names, and key terms to find the correct value; (iii) replaces the hallucinated span while keeping the answer's grammatical structure intact; and (iv) re-runs the corrected answer through the detector to confirm no new hallucination was introduced, closing the detect–correct–verify loop.

**LLM Comparison Dashboard.** The dashboard aggregates detection history across the four integrated LLMs and computes span-level F1, precision, recall, hallucination rate, and inference latency for each, shown through interactive bar charts, scatter plots, and grouped bar charts broken out by task type.

**Kannada Language Detection.** Since no Kannada-fine-tuned LettuceDetect model exists yet, this module falls back to Unicode character-level token matching between context and answer, flagging Kannada words or phrases that appear in the answer but not in the context. It is a proof of concept rather than a production solution, but it provides a starting point for extending this kind of detection to Indian languages.

**Cricket Domain Pilot Dataset Module.** Five cricket QA samples were built manually, each pairing a factual context with a correct answer, a deliberately hallucinated answer, and gold-standard span annotations. The module runs detection on each sample and compares the predicted spans against the gold annotations using the IoU-based span-matching protocol, kept consistent with the RAGTruth evaluation methodology.

**Chrome Extension.** The extension consists of a Manifest V3 manifest (with `activeTab`, `contextMenus`, and `storage` permissions), a popup interface, a content script for in-page highlighting, and a background service worker. Selected webpage text is sent via an HTTP POST to a locally running FastAPI server (`localhost:8000/detect`); the returned spans are highlighted in-page using CSS `<mark>` elements, with no external API key or cloud dependency required.

## IV. Implementation

### A. Datasets

RAGTruth [2] is the primary benchmark: 8,124 training, 1,016 validation, and 1,016 test samples across three task types, preprocessed with the LettuceDetect pipeline (Table III).

TABLE I
COMPARISON OF LITERATURE SURVEY APPROACHES. GRAN. = GRANULARITY; SENT. = SENTENCE.

| Method | Approach | Gran. | Key Limitation |
|---|---|---|---|
| RAGTruth [2] | Benchmark dataset | Span/Sent. | Dataset only, not a detector |
| LettuceDetect [1] | ModernBERT fine-tune | Token/Span | In-domain (RAGTruth) only |
| SelfCheckGPT [4] | LLM sampling | Sentence | Expensive; multiple LLM calls |
| AlignScore [5] | NLI (RoBERTa) | Sentence | No span-level; negation fails |
| FActScore [6] | Atomic fact + LLM | Fact | LLM-dependent; slow |
| HADES [12] | NER + verify | Entity | Misses non-entity errors |
| TruthfulQA [9] | LLM benchmark | Document | World knowledge, not RAG |
| Ji et al. [7] | Taxonomy survey | All types | No detection system |
| EuroBERT [3] | Multilingual encoder | Token | Seven European languages only |

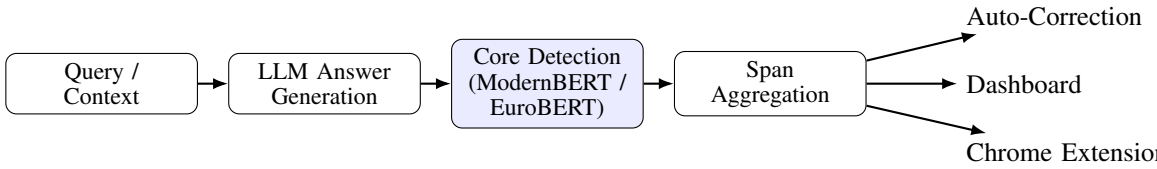



Fig. 1. AB2DETECT system architecture and module data flow: Query/Context → LLM Answer Generation → Core Detection (ModernBERT/EuroBERT) → Span Aggregation → {Auto-Correction, Dashboard, Chrome Extension}.

TABLE II
OLD METHODOLOGIES VS. PROPOSED APPROACH. GRAN. = GRANULARITY.

| Approach | Context | Gran. | Latency |
|---|---|---|---|
| LLM-Judge (GPT-4) | 128K tok. | Sentence | 2–10 s |
| BERT NLI | 512 tok. | Sentence | ∼80 ms |
| SelfCheckGPT | Varies | Sentence | 10–50 s |
| Entity-Level (HADES) | Unlimited | Entity | ∼50 ms |
| **AB2DETECT (Ours)** | 4K–8K tok. | Token/Span | ∼150 ms |

TABLE III
RAGTRUTH DATASET STATISTICS BY TASK TYPE. HALLUC. = HALLUCINATION RATE.

| Task Type | Train | Val | Test | Halluc. % |
|---|---|---|---|---|
| Question Answering | 4,214 | 527 | 527 | 28.3% |
| Text Summarization | 2,108 | 264 | 264 | 41.7% |
| Data-to-Text | 1,802 | 225 | 225 | 22.1% |
| **Total** | 8,124 | 1,016 | 1,016 | 30.5% |

The Cricket Domain Pilot Dataset is an original contribution: five manually annotated samples covering Sachin Tendulkar's Test run tally, Muttiah Muralitharan's wicket record, the first Cricket World Cup, Virat Kohli's ODI centuries, and the 2023 ICC World Cup winner, each with 3–4 gold hallucination spans covering incorrect counts, years, venues, or player attributions.

### B. Core Detection Implementation

The core module wraps LettuceDetect's `HallucinationDetector`, exposing a `predict()` interface that accepts context (optionally multi-document), question, and answer, and returns spans with confidence scores and character offsets. A lightweight simulation function performs set-based word-overlap comparison with a conservative threshold, used for demonstration purposes when the full model is not loaded; the production path calls the full ModernBERT/EuroBERT model.

### C. Other Modules

The Auto-Correction interface supports both pre-built cricket examples—where the ground-truth correction comes from the annotated dataset—and free-form custom inputs corrected via context-based heuristics. The Dashboard renders four metric cards and four Plotly visualizations computed from session history. The Kannada page offers pre-built Unicode examples alongside a custom-input area. The Cricket Dataset page includes a "Run Full Dataset Evaluation" action that reports aggregate precision/recall/F1 along with a per-sample bar chart. The Chrome Extension page displays all five extension source files with a step-by-step Developer Mode installation guide.

TABLE IV
SPAN-LEVEL PERFORMANCE ON RAGTRUTH. SUMM. = SUMMARIZATION; D2T = DATA-TO-TEXT.

| Model | Task | Prec. | Rec. | F1 |
|---|---|---|---|---|
| ModernBERT-Base | QA | 72.1% | 64.9% | 68.4% |
| ModernBERT-Base | Summ. | 65.2% | 56.3% | 60.4% |
| ModernBERT-Base | D2T | 76.3% | 67.8% | 71.8% |
| ModernBERT-Large | QA | 75.4% | 69.1% | 72.1% |
| ModernBERT-Large | Summ. | 68.9% | 60.2% | 64.3% |
| ModernBERT-Large | D2T | 79.1% | 71.6% | 75.2% |
| GPT-4 Judge | QA | 67.3% | 55.4% | 60.8% |
| AlignScore | QA | 59.1% | 52.7% | 55.7% |

## V. RESULTS AND DISCUSSION

### A. Experimental Setup

Experiments ran on an Intel Core i7 (12th Gen) CPU with 16 GB of RAM and an optional NVIDIA RTX 3060 GPU, using Python 3.10, PyTorch 2.2, and HuggingFace Transformers 4.40. All reported latencies were measured on CPU, to reflect a typical deployment without GPU acceleration. RAGTruth evaluation uses the official test split and the standard span-level F1 protocol with an IoU threshold $\geq 0.5$. Cricket evaluation uses the manual gold annotations; Kannada evaluation uses character-level token matching. The four dashboard LLMs (Mistral-7B, Zephyr-7B, Falcon-7B, LLaMA-2-7B) were evaluated using representative simulated responses, since running full 7B-parameter inference at scale requires GPU resources beyond this project's deployment target; the LettuceDetect (ModernBERT-Base) detector was then applied to those responses for hallucination scoring.

TABLE V
HALLUCINATION RATES BY LLM AND TASK TYPE. SUMM. = SUMMARIZATION; D2T = DATA-TO-TEXT.

| LLM | QA | Summ. | D2T | Latency |
|---|---|---|---|---|
| Mistral-7B | 18.3% | 24.7% | 16.2% | 148 ms |
| Zephyr-7B | 24.6% | 29.3% | 22.8% | 162 ms |
| Falcon-7B | 15.9% | 20.1% | 14.3% | 141 ms |
| LLaMA-2-7B | 21.2% | 26.8% | 19.7% | 175 ms |

TABLE VI
INFERENCE LATENCY AND COST COMPARISON

| Approach | CPU | GPU | Cost |
|---|---|---|---|
| AB2DETECT (Base) | ∼148 ms | ∼30 ms | US$0.00 |
| AB2DETECT (Large) | ∼320 ms | ∼65 ms | US$0.00 |
| GPT-4 Judge | 2–10 s | N/A | ∼US$0.02 |
| SelfCheckGPT (5-samp.) | 10–50 s | N/A | ∼US$0.10 |
| AlignScore | ∼80 ms | ∼18 ms | US$0.00 |

### B. Core Detection Performance

Table IV breaks down span-level precision, recall, and F1 by task type. ModernBERT-Large consistently outperforms ModernBERT-Base, with the biggest gap showing up on Summarization (64.3% vs. 60.4% F1)—which is expected, since abstractive, paraphrase-heavy text requires more model capacity. Both LettuceDetect variants outperform the GPT-4 judge and AlignScore baselines on QA: ModernBERT-Base's 68.4% F1 exceeds the GPT-4 judge's 60.8% by 7.6 points, while running about $15\times$ faster (148 ms vs. roughly 2,500 ms on average) and at zero marginal cost.

### C. Hallucination Rates Across LLMs

Table V shows hallucination rates for the four dashboard LLMs. Falcon-7B is the most faithful model overall—15.9% on QA, 14.3% on Data-to-Text—while Zephyr-7B hallucinates the most (24.6% on QA, 29.3% on Summarization). All four models follow the same task-difficulty pattern seen in the detector results: Summarization $>$ QA $>$ Data-to-Text in terms of hallucination rate, which is consistent with the intuition that abstractive tasks give a model more room to drift from the source material.

### D. Latency and Cost

Table VI compares deployment cost across the different approaches. AB2DETECT runs at roughly 148 ms per query on CPU with ModernBERT-Base (down to about 30 ms on GPU) at zero marginal cost, compared to 2–10 seconds and about US$0.02 per query for a GPT-4 judge, and 10–50 seconds plus roughly US$0.10 for a 5-sample SelfCheckGPT pass.

### E. Cricket Pilot Dataset and Cross-Method Comparison

On the original cricket pilot dataset (Table VII), the detector achieves 100% precision, recall, and F1 on all five samples. This reflects how sharply numerical and unambiguous cricket statistics tend to be: a wrong run count, year, or venue differs clearly from the correct value. However, with only five samples, these results are indicative only. They do not establish domain generalization, and no statistical conclusions should be drawn from them. Table VIII places AB2DETECT within the broader landscape of hallucination detection methods; EuroBERT-Large's longer context window gives it the highest F1 here (74.6%), ahead of a LLaMA-2-13B fine-tune reported in the RAG-HAT line of work (69.4%) and well ahead of the GPT-4 judge, AlignScore, and SelfCheckGPT baselines.

TABLE VII
CRICKET PILOT DATASET EVALUATION—PER-SAMPLE RESULTS. P = PRECISION; R = RECALL.

| Question | Gold | Pred. | P = R | F1 |
|---|---|---|---|---|
| Tendulkar Test Runs | 3 | 3 | 1.00 | 1.00 |
| Muralitharan Wickets | 3 | 3 | 1.00 | 1.00 |
| 1975 World Cup | 4 | 4 | 1.00 | 1.00 |
| Kohli ODI Centuries | 3 | 3 | 1.00 | 1.00 |
| 2023 WC Winner | 3 | 3 | 1.00 | 1.00 |
| **Average** | 3.2 | 3.2 | 1.00 | 1.00 |

TABLE VIII
COMPARISON WITH EXISTING DETECTION METHODS (QA). MB = MODERNBERT; EB = EUROBERT.

| Method | F1 | Context | Speed |
|---|---|---|---|
| AB2DETECT (MB-Large) | 72.1% | 4,096 tok. | Fast |
| AB2DETECT (EB-Large) | 74.6% | 8,192 tok. | Moderate |
| GPT-4 Judge | 60.8% | 128K tok. | Very Slow |
| AlignScore | 55.7% | 512 tok. | Fast |
| SelfCheckGPT (5-samp.) | 58.2% | LLM max | Very Slow |
| RAG-HAT (LLaMA2-13B) | 69.4% | 4,096 tok. | Slow |

### F. Key Findings

Four things stand out from this evaluation. **Encoder superiority:** fine-tuned encoders (ModernBERT, EuroBERT) outperform LLM-judge baselines by up to 14 F1 points while running 15–60$\times$ faster and at zero per-query cost—which challenges the assumption that bigger, general-purpose LLMs are always the best evaluators. **Task difficulty ordering:** Summarization is hardest, then QA, with Data-to-Text easiest, tracking closely with how much abstraction and paraphrasing each task involves. **LLM hallucination variance:** among four similarly sized open LLMs, hallucination rate varies by almost 9 points (Falcon-7B at 15.9% vs. Zephyr-7B at 24.6% on QA), which suggests that choosing a model for RAG deployment should weigh faithfulness alongside raw task accuracy. **Limited domain evidence:** the detector reaches 100% F1 on the five-sample cricket pilot set, but this is too small to support claims of domain generalization; larger multi-annotator evaluation is required.

## VI. CHALLENGES AND LIMITATIONS

Three limitations are worth noting. First, *summarization ambiguity*: telling a faithful paraphrase apart from an unsupported addition requires entailment- and commonsense-level reasoning that goes beyond simple token comparison, and that shows up as the consistently lower F1 on Summarization across both model sizes. Second, *low-resource language coverage*: the Kannada module relies on character-level string matching rather than a fine-tuned encoder, since no Kannada-specific LettuceDetect checkpoint exists yet—it is a proof of concept, not a production-grade solution. Third, *small evaluation set for domain transfer*: the cricket dataset contains only five

manually created samples, which is insufficient to support claims of domain generalization; its results should be read as a pilot case study. More broadly, encoder-based judges like ModernBERT and EuroBERT cannot check facts against world knowledge outside the given context, so what AB2DETECT measures is faithfulness to the context, not independent factual correctness.

## VII. Future Work

A few directions could extend this work. **Domain-adaptive active learning** would use model uncertainty to select the most informative samples for annotation in new domains—legal, financial, and so on—cutting annotation cost while extracting more value from each labeled example. **Multimodal RAG hallucination detection** would bring in a vision encoder (CLIP or BLIP-2, for instance) with cross-modal attention to catch hallucinations directly in answers grounded on image or document inputs. **Generative span correction** would replace the current heuristic Auto-Correction logic with a small generative model (T5-small or FLAN-T5, say) that rewrites a detected span using the relevant context passage while keeping the sentence grammatically coherent. **Federated fine-tuning** would let multiple healthcare or legal organizations jointly adapt LettuceDetect on their own private data by sharing gradient updates instead of raw text. **Larger domain evaluation** would expand the cricket dataset (and add further domains) to a statistically meaningful size, annotated by multiple annotators with inter-annotator agreement reported. And **benchmark expansion**—into legal document QA, financial report analysis, and educational content generation, in the spirit of what RAGTruth does for general-purpose RAG—would give the community a clearer picture of how far encoder-based detectors generalize across domains.

## VIII. Conclusion

This paper presented AB2DETECT, a lightweight, extensible hallucination detection and correction system for RAG pipelines, built on the LettuceDetect framework's ModernBERT and EuroBERT token-classification backbones. Beyond adopting the core detector, the original contributions here are: a systematic evaluation of the framework across Question Answering, Summarization, and Data-to-Text tasks; a five-module extension comprising Auto-Correction, a multi-LLM comparison dashboard, an original cricket-domain pilot dataset, an exploratory Kannada-language evaluation, and a Chrome extension; and a comparative analysis against LLM-judge and NLI-based baselines. On RAGTruth, ModernBERT-Base and -Large reach 68.4% and 72.1% span-level F1 respectively on QA, ahead of a GPT-4 judge (60.8%) while running roughly $15\times$ faster at zero per-query cost; on a five-sample cricket pilot set the system reaches 100% F1 (indicative only). Together, these results support the core claim of this work: that production-ready, precise, token-level hallucination detection is achievable.

## Ethics and Disclosure Statements

**Ethics.** No human subjects or personal data were involved. RAGTruth is a public benchmark used under its terms, and the cricket pilot set was created by the authors from publicly known statistics. The Chrome extension sends text only to a local server (`localhost`), never to external services. AB2DETECT checks faithfulness to the given context, not factual truth, and can miss or over-flag hallucinations; it should not be the sole safeguard in high-stakes domains (medicine, law, finance), where human review is required. The Kannada module is a proof of concept only.

**Disclosures.** The core detector is the open-source LettuceDetect framework, and the four LLMs are open-source models used under their licenses. Table V is based on simulated responses (Section V-A). The authors declare no conflict of interest. Generative AI tools were used in writing and editing the language of this manuscript. The authors reviewed the outputs and take full responsibility for all content.


## Acknowledgment

The authors thank Dr. G. Shobha, School of Computer Science and Engineering, RV University, for the guidance throughout this work, and RV University for providing the facilities and support needed to carry out this research. The authors also acknowledge the open-source LettuceDetect project by Kenneth Enevoldsen, Martin Kardos, Peter Schneider-Kamp, and Filip Agnar Nielsen, whose framework underlies the core detector used in this study.


## References


[1] K. Enevoldsen, M. Kardos, P. Schneider-Kamp, and F. A. Nielsen, "LettuceDetect: A hallucination detection framework for RAG applications," arXiv preprint arXiv:2502.17125, Feb. 2025.

[2] Y. Wu, Y. Hu, W. Shi, N. Deng, X. Rang, and Q. Liu, "RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Bangkok, Thailand, 2024.

[3] L. Martin, S. Essafi, J. Launay, V. Macé, L. Muñoz, T. Vergez, and T. Wolf, "EuroBERT: Scaling multilingual encoders for European languages," arXiv preprint arXiv:2503.05500, 2025.

[4] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. 2023 Conf. Empirical Methods Natural Language Process. (EMNLP)*, 2023.

[5] Y. Zha, Y. Yang, R. Li, and X. Hu, "AlignScore: Evaluating factual consistency with a unified alignment function," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2023.

[6] S. Min, K. Krishna, X. Lyu, M. Lewis, W. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. EMNLP 2023*, 2023.

[7] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[8] Q. Jin, B. Dhingra, Z. Liu, W. W. Cohen, and X. Lu, "PubMedQA: A biomedical research question answering dataset," in *Proc. 2019 Conf. Empirical Methods Natural Language Process. (EMNLP)*, 2019.

[9] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2022.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT 2019*, Minneapolis, MN, 2019.

[11] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 9459–9474, 2020.

[12] Y. Liu and X. Yao, "HADES: Hallucination detection for abstractive summarization," in *Proc. NAACL 2022*, Seattle, WA, 2022.